

Sección A. Preparación Específica del Entorno de Red y Sistema Operativo:

A1. Inicialización y preparación:

Para la Máquina Virtual Local (VirtualBox - Ubuntu Server 22.04 LTS):

Inicializar la máquina virtual:

Se inicializa la instancia en VirtualBox asignando recursos base optimizados para el servicio multimedia: Tipo: Linux, Versión: Ubuntu (64-bit), 2048 MB de RAM y un disco virtual de 20 GB.

Para la Instalación y Despliegue para AWS EC2 Amazon - Ubuntu Server 24.04 LTS (Nube):

Configuración de Red Externa: Se asigna una IP Elástica (estática pública) a la instancia EC2 y se configuran los registros en el proveedor de DNS para asociar el nombre de dominio con el servidor en la nube. Asimismo, se abre el puerto 4533 en el Security Group de AWS. Además de las reglas bases, también se configura la siguiente regla con los siguientes datos:

Type (Tipo): Custom TCP (TCP personalizado)

Port range (Rango de puertos): 4533

Source (Origen): Selecciona Anywhere-IPv4 (Cualquier lugar IPv4).(Mi IP)

En la primera parte de esta sección, los comandos son los mismos para ambos entornos.

A2. Actualización de Repositorios y Paquetes :

```
sudo apt update && sudo apt upgrade -y
```

Actualiza la lista de paquetes disponibles e instala las actualizaciones del sistema antes de comenzar la instalación de nuevas herramientas.

A3.Instalación del Motor de Contenedores/ Podman en la Nube:

```
sudo apt install -y podman
```

Instala el motor de contenedores Podman desde los repositorios oficiales de Ubuntu o desde los repositorios oficiales optimizados de AWS.

A4.Verificar instalación:

```
podman --version
```

Comprueba que Podman se instaló correctamente y que puede ejecutarse desde la terminal.

A5.Verificar que funciones sin privilegios de root:

```
podman info | grep -i rootless
```

Verifica que Podman esté funcionando en modo rootless, evitando que el contenedor requiera privilegios de administrador.

A6.Creación del Árbol de Directorios:

```
mkdir -p ~/navidrome/data ~/navidrome/music
```

Crea las carpetas donde se almacenarán la biblioteca musical y los datos generados por la aplicación. -p (parents) para indicarle al sistema operativo que cree automáticamente las carpetas superiores si es que aún no existen

A7. Guardar base de datos (solo en VirtualBox):

```
podman volume create navidrome-data
```

Genera un volumen administrado por Podman para guardar la base de datos y la configuración del servicio.

A8. Inicialización y Despliegue del Contenedor:

```
podman run -d --name navidrome -p 4533:4533 -v ~/navidrome/music:/music:ro -v navidrome-data:/data --restart unless-stopped docker.io/deluan/navidrome:latest
```

Descarga la imagen oficial de Navidrome (si aún no está disponible) e inicia el contenedor en segundo plano. Además, configura el puerto 4533, monta la carpeta de música en modo de solo lectura y enlaza el volumen destinado a la base de datos.

A9. Control del Estado del Proceso:

```
podman ps
```

```
podman logs navidrome
```

Muestra los contenedores que se encuentran en ejecución y permite comprobar que Navidrome está activo. Inspecciona la salida estándar y de errores del contenedor para auditar el correcto inicio del servidor embebido de Navidrome y asegurar que no existan bloqueos de I/O en la apertura de la base de datos SQLite.

A10. Instrumentación de Red a Bajo Nivel:

```
ss -tulpn | grep 4533
```

Confirma que el servicio está escuchando en el puerto 4533 y listo para recibir conexiones.

A11. Identificación de Direccionamiento IP:

```
hostname -I
```

Muestra la dirección IP asignada a la máquina virtual para acceder a la aplicación desde otro equipo de la red.

A12. Crear el usuario administrador (solo en VirtualBox - Ubuntu Server 22.04 LTS):

Se accede mediante un navegador web en el host a la dirección `http://IP_DE_LA_VM:4533` para dar de alta la cuenta del administrador general.

Al ingresar por primera vez, Navidrome solicita la creación de la cuenta de administrador. En AWS, como se tiene el dominio enlazado con nuestro terminal, solo utilizamos la dirección de nuestro dominio.

Sección B — Acceso desde el navegador de Windows (solo en VirtualBox - Ubuntu Server 22.04 LTS):

B1. Apagar la VM por completo:

VirtualBox → clic derecho en la VM → Apagar.

B2. Abrir la configuración de red:

VirtualBox → seleccionar la VM → Configuración → Red → Avanzadas → Reenvío de puertos.

B3. Agregar la regla de reenvío:

Protocolo: TCP | IP anfitrión: (vacío) | Puerto anfitrión: 4533 | IP invitado: 10.0.2.15 | Puerto invitado: 4533

Dado que la interfaz NAT por defecto encapsula la IP de la máquina virtual, se implementó una regla síncrona de reenvío de puertos mediante la interfaz de administración de VirtualBox.

B4. Encender la VM nuevamente:

VirtualBox → Iniciar.

B5. Acceder desde el navegador de Windows:

`http://IP_DE_LA_VM:4533`

En la consola de AWS EC2, se asoció una IP elástica (estática pública) a la instancia y se alteró la tabla de directivas de seguridad en el firewall perimetral de AWS (Security Group), añadiendo una regla de entrada (Inbound Rule).

Sección C. Operación, Control de Estado y Administración del Ciclo de Vida:

Una vez desplegado el contenedor, se establecieron procedimientos operativos estándar, para supervisar procesos secundarios, analizar logs en tiempo real y restaurar accesos

administrativos mediante la interacción directa con los binarios internos, garantizando la continuidad del servicio de streaming (tanto para la VM de VirtualBox como para el servidor de AWS EC2-Amazon):

C1. Ver el estado de todos los contenedores:

```
podman ps -a
```

Inspección global del estado de salud y códigos de salida (Status Up/Exited).

C2. Reactivación del contenedor:

```
podman start navidrome
```

Reactivación síncrona del contenedor ante paradas controladas del sistema operativo

C3. Confirmar la activación:

```
podman ps
```

```
podman logs navidrome
```

Extracción de la salida estándar (Stdout) y auditoría de logs del servidor web interno

C4. Ver a qué puerto está mapeado:

```
podman port navidrome
```

C5. Entrar desde el navegador:

http://<IP_de_la_VM>:4533 o <http://localhost:4533/> (propia VM)

http://<IP_del_servidor>:4533

dominio(<http://sportiafai.xyz/app/#/album/69lwB2p7tQDejD3lowUIFo/show>) -> AWS

o

Sección D. Recuperar acceso (usuario / contraseña):

D1. Ver la lista de usuarios existentes

```
podman exec navidrome navidrome user list
```

Muestra el username exacto.

D2. Cambiar la contraseña de ese usuario

```
podman exec -it navidrome navidrome user edit --user Ope2026 --set-password
```

El flag -it permite escribir la nueva contraseña de forma interactiva cuando el sistema la solicite.

D3. Iniciar sesión con la nueva contraseña.

Sección E. Ingesta de Datos Multimedia, Autenticación mediante Cookies y Pruebas de Streaming vía Terminal:

Para probar el rendimiento de lectura y escritura (I/O) con una carga de trabajo real, automatizamos la descarga de audio desde plataformas de streaming utilizando yt-dlp. Al hacer esto en la nube (AWS EC2), nos encontramos con que YouTube limitaba la velocidad o nos bloqueaba por la reputación de la IP. Para evitarlo, usamos Deno e incluimos cookies de sesión de un usuario real (exportadas desde el navegador con la extensión 'Get cookies.txt LOCALLY'). Una vez descargados los archivos, ajustamos los permisos de seguridad de forma estricta para que el usuario sin privilegios de Podman pudiera leerlos sin problema, y terminamos forzando un escaneo manual en la aplicación.

E1. Instalar las herramientas:

=====Exclusivo
para VirtualBox - Ubuntu Server 22.04 LTS:

```
sudo apt update && sudo apt install -y ffmpeg pipx unzip python3-pip
```

```
sudo apt update && sudo apt install -y ffmpeg python3-pip
```

```
sudo pip3 install yt-dlp
```

```
yt-dlp --version
```

=====

AWS EC2 Amazon - Ubuntu 24.04 LTS:

```
sudo apt update && sudo apt install -y ffmpeg pipx unzip python3-pip
pipx install "yt-dlp[default,plugins]" && pipx ensurepath && source ~/.bashrc
yt-dlp --version
```

*Motor de JavaScript (Deno):

```
curl -fsSL https://deno.land/install.sh | sh
```

*Enlazar Deno globalmente en el sistema:

```
sudo ln -s /home/ubuntu/.deno/bin/deno /usr/local/bin/deno
```

*Configuración del almacén de credenciales de sesión (Pegar contenido de Get cookies.txt LOCALLY)/En la terminal de AWS, crea/actualiza el archivo:

```
rm -f cookies.txt
```

```
nano cookies.txt
```

E2. Descarga Secuencial, Evasión de Bloqueos y Conversión a Alta Fidelidad:

=====Exclusivo
para VirtualBox - Ubuntu Server 22.04 LTS (canciones ya cargadas):

```
yt-dlp -x --audio-format mp3 --audio-quality 0 -o
"/home/ope2026/navidrome/music/OPE/%(title)s.%(ext)s"
"https://youtu.be/9FRAhDQsek8?si=09pd8x9OUqNebVSG"
"https://youtu.be/29bfaHc1XiY?si=QpHGOPdSH5L7FO0t"
"https://youtu.be/oE56g61mW44?si=IXsTD9fDKPWDjg_X"
```

AWS EC2 Amazon - Ubuntu 24.04 LTS (canciones ya cargadas):

```
yt-dlp --extractor-args "youtube:player-client=web,tv" --cookies cookies.txt -x --audio-format
mp3 --audio-quality 0 -o "/home/ubuntu/navidrome/music/OPE/%(title)s.%(ext)s"
"https://youtu.be/Zi_XLOBD0_Y?si=J6vor462zrVfHyis"
"https://youtu.be/29bfaHc1XiY?si=QpHGOPdSH5L7FO0t"
"https://youtu.be/oE56g61mW44?si=IXsTD9fDKPWDjg_X"
"https://youtu.be/9FRAhDQsek8?si=yp9WvqF9loZuZuFS"
```

Se inyectan los argumentos de simulación de cliente web/TV y las cookies cargadas.
Procedimiento de depuración física de bloques .

```
rm "/home/ubuntu/navidrome/music/OPE/nombre_de_la_cancion.mp3"
```

E3. Asegurar permisos para Podman:

=====Exclusivo
para VirtualBox - Ubuntu Server 22.04 LTS:

```
chmod -R 755 ~/navidrome/music/
```

AWS EC2 Amazon - Ubuntu 24.04 LTS:

```
sudo chown -R ubuntu:ubuntu /home/ubuntu/navidrome/music/
```

```
chmod -R 755 /home/ubuntu/navidrome/music/
```

Configuración estricta de propiedad y permisos de lectura/ejecución para aislamiento Rootless.

E4. Verificar que los archivos se descargaron:

=====Exclusivo
para VirtualBox - Ubuntu Server 22.04 LTS:

```
ls -lh ~/navidrome/music/OPE/
```

AWS EC2 Amazon - Ubuntu 24.04 LTS:

```
ls -lh /home/ubuntu/navidrome/music/OPE/
```

Inspección de las estructuras de archivos asignadas en el sistema de archivos del Host.

E5. Forzar que Navidrome escanee la nueva música:

```
podman exec -it navidrome /app/navidrome scan
```

Navidrome también escanea automáticamente, entre 1 a 2 minutos de espera.

Sección F. Reproductor de Música:

Para medir con precisión el tiempo de respuesta del servidor de música Navidrome, se diseñó un método de prueba que evita el uso de navegadores web comunes, ya que los reproductores comerciales guardan datos en la memoria caché, lo que altera las mediciones reales de velocidad.

En su lugar, las solicitudes de audio se realizan directamente desde la terminal de comandos utilizando la herramienta curl. De esta forma, se interactúa directamente con la API REST de Navidrome, simulando la descarga del flujo de música puro y sin intermediarios.

F1. Listar los archivos dentro del contenedor

```
podman exec -it navidrome ls -lh /music/OPE
```

Muestre la lista de canciones que tiene para escanear y reproducir.

F2. Buscar el ID de la canción en Navidrome:

```
sudo apt update && sudo apt install -y curl
```

```
curl -s 'http://localhost:4533/rest/search3?query=Shawn&u=Ope2026&p=OPE2026&v=1.16.1&c=miapp&f=json'
```

====

AWS

```
curl -s "http://localhost:4533/rest/search3?u=Ope2026&p=ope2026&v=1.16.1&c=miapp&query=Electric"
```

Busca el campo "id" de la canción.

F3. Pedir el stream de esa canción:

```
sudo apt install mpv -y
```

```
curl -s "http://localhost:4533/rest/stream?id=8WkBzWe2j0wZWCToebLpwh&u=Ope2026&p=ope2026&v=1.16.1&c=miapp" -o /dev/null *(silenciosos) -> Funciona en ambos espacios.
```

mpv

```
"http://localhost:4533/rest/stream?id=8WkBzWe2j0wZWCToebLpwhO&u=Ope2026&p=ope2026&v=1.16.1&c=miapp" *(cachè) -> Funciona en Linux/MV
```

```
mpv --no-ytdl --cache=no --demuxer-max-bytes=100 "http://localhost:4533/rest/stream?id=7fH11doysdXoRqQvg9zVTs&u=Ope2026&p=OPE2026&v=1.16.1&c=miapp"(tiempo real)
```

mplayer -ao null -> Funciona en Linux/MV

```
curl -I "http://localhost:4533/rest/stream?id=Z9u7FLdVxUqAZ4MKwof7nX&u=Ope2026&p=ope2026&v=1.16.1&c=miapp" -> Funciona en AWS
```

*Escuchar el archivo en la terminal de forma visual(AWS no tiene una tarjeta de sonido física para reproducir audio directamente en el aire).

Conecta la terminal directamente al flujo de audio de Navidrome. Al ejecutarlo, la canción comenzará a sonar inmediatamente a través de los altavoces de tu sistema.

F4. Confirmar visualmente que Navidrome sirvió la petición:

```
podman logs --tail 20 navidrome
```

Auditoría del búfer de salida del proceso del ciclo de vida del contenedor para confirmar la petición.

Sección G. Aplicar límites de recursos (cgroups):

Demostrar los mecanismos de control que el sistema operativo ejerce sobre los entornos de aislamiento. Se contemplaron dos enfoques metodológicos: la modificación dinámica en caliente mediante las propiedades de unidad de Systemd y la recreación estática inmutable del contenedor. El establecimiento de un límite agresivo (128 MB) tiene el objetivo técnico de forzar al Kernel a desalojar la caché de páginas de forma acelerada, permitiendo registrar la degradación del rendimiento en las fases de experimentación posteriores.

Opción 1: Reconfiguración Dinámica de Atributos del Scope en Caliente (Systemd Cgroups)

G1. Inspección del identificador completo del contenedor para determinar su firma en el Host:

```
podman ps --no-trunc
```

```
podman ps
```

Encontrar el PID.Se necesitas para saber el nombre exacto de la "unidad" de Systemd que lo controla.

G2. Aplicar el límite de memoria dinámicamente vía Systemd:

```
systemctl --user set-property libpod-$(podman inspect navidrome --format '{{.ID}}').scope MemoryMax=128M (128 Megabytes)
```

```
systemctl --user set-property libpod-$(podman inspect navidrome --format '{{.ID}}').scope CPUQuota=100% (100% de 1 núcleo)
```

Asignación de límites físicos instantáneos en la jerarquía cgroups del usuario sin interrumpir el proceso

G3. Confirmar que el límite se aplicó en el contenedor:

```
systemctl --user show libpod-$(podman inspect navidrome --format '{{.ID}}').scope | grep -E "MemoryMax|CPUQuota"
```

Validación de la persistencia temporal de los parámetros asignados por el subsistema de control.

Opción 2: Parámetros de Planificación (Recomendada):

G1. Inspección del identificador completo del contenedor para determinar su firma en el Host:

```
podman stop navidrome && podman rm navidrome
```

Destrucción controlada de la instancia sin pérdida de persistencia gracias a los volúmenes externos podman stop navidrome && podman rm navidrome.

G2. Despliegue de la arquitectura bajo límites agresivos de recursos del Kernel (Asegurando rutas absolutas completas) :

```
podman run -d --name navidrome \
--memory="128m" \
--memory-swap="128m" \
-p 4533:4533 \
-v ~/navidrome/music:/music:ro \
-v ~/navidrome/data:/data \
docker.io/deluan/navidrome:latest
```

Un límite agresivo (128m) fuerza al kernel a desalojar la caché de página más rápido, lo cual es justo lo que necesitas para notar diferencia en el experimento.

Sección H. Instrumentación Interna del Kernel de Linux y Captura de Evidencia de Telemetría:

Esta fase metodológica proporciona la visibilidad requerida para auditar las estructuras lógicas de bajo nivel asignadas por el Kernel de Ubuntu Server para controlar los procesos del contenedor. La ejecución sincronizada de estos comandos bajo cargas activas de streaming multimedia constituye el núcleo de la instrumentación del sistema operativo.

H1. Ver los procesos internos del contenedor (sin daemon root):

```
podman top navidrome
```

Mapeo en vivo de los hilos de ejecución internos del contenedor y usuarios del sistema.

H2. Obtener el PID real del proceso principal:

```
podman inspect navidrome --format '{{.State.Pid}}'
```

Inspección de las variables de estado para capturar el PID real asignado por el Planificador del Kernel. Anotar PID.

H3. Ver el estado del proceso desde /proc:

```
cat /proc/[PID]/status
```

Extracción de la huella de memoria y variables de ejecución desde el pseudo-sistema de archivos del Host. Reemplazar [PID] obtenido en el paso anterior.

H4. Ver namespaces aislados del contenedor

```
lsns -p [PID]
```

Auditoría de los Namespaces aislados del Kernel (Red, Montaje, IPC, CPU) asociados a dicho PID.

H5. Ver el estado del proceso desde /proc:

```
cat
```

```
/sys/fs/cgroup/user.slice/user-1000.slice/user@1000.service/user.slice/libpod-0d047f301438660fd5f98e999dd7084edabbae48f2daf4c0c41bb000df4f8f1f.scope/memory.max -> MV
```

Interrogación Directa a los Subsistemas y Contadores Físicos de Cgroups v2.

H6. Ver el uso real de memoria del cgroup:

```
cat /sys/fs/cgroup/user.slice/user-$(id -u).slice/user@$(id -u).service/user.slice/libpod-$(podman inspect navidrome --format '{{.ID}}').scope/memory.current
```

H7. Ver cuánta de esa memoria es caché de página:

```
cat /sys/fs/cgroup/user.slice/user-$(id -u).slice/user@$(id -u).service/user.slice/libpod-$(podman inspect navidrome --format '{{.ID}}').scope/memory.stat
```

Análisis discriminatorio de la memoria (Permite aislar el parámetro "file" correspondiente a la Page Cache asignada).

H8. Ver métricas en vivo (CPU, RAM, I/O de bloque)

```
podman stats navidrome --no-stream
```

Captura de la telemetría global instantánea del subsistema de contenedores de Podman.

Sección I. Metodología del Experimento de Page Cache, Buffering y Recolección de Datos :

Para corroborar empíricamente que la aceleración del procesamiento secuencial de archivos multimedia es gestionada por los algoritmos del Kernel de Linux y no por la capa de aplicación, se diseñó un experimento controlado enfocado en la medición del tiempo de Entrada/Salida bajo dos estados lógicos de la memoria RAM del sistema:

- **Simulación de Lectura en Frío (Cold Cache):** Se forzó la liberación completa de la memoria caché de páginas del subsistema virtual del Kernel, obligando a realizar una llamada de E/S síncrona en el hardware físico de almacenamiento de bloques.
- **Simulación de Lectura en Caliente (Warm Cache):** Se repitió síncronamente la misma operación de lectura contigua de bloques binarios sin alterar las cachés, midiendo el salto de velocidad provocado por la persistencia del archivo en la RAM del Host.

Paso 1: Evaluación Base de Tiempos de Acceso Síncronos (Sin restricciones de cgroups)

I1. Vaciar la caché de página del sistema (requiere sudo):

```
sudo sync; echo 3 | sudo tee /proc/sys/vm/drop_caches
```

Sincronización de operaciones pendientes en disco y vaciado forzado de la Page Cache del Kernel. Esto obliga a que la próxima lectura sea forzosamente desde el disco, no desde RAM.

I2. Medir la lectura en FRÍO (primera vez, desde disco)

```
time dd if="/home/ope2026/navidrome/music/OPE/Shawn Mendes - Treat You Better (Live from New York).mp3" of=/dev/null bs=1M
```

=====

AWS:

```
time dd if="/home/ubuntu/navidrome/music/OPE/Isn't She Lovely.mp3" of=/dev/null bs=1M
```

Fase 1: Medición de velocidad en frío mediante copia binaria secuencial directa a un sumidero virtual.

I3. Medir la lectura en CALIENTE (repetir el mismo comando, sin vaciar caché)

```
time dd if="/home/ope2026/navidrome/music/OPE/Shawn Mendes - Treat You Better (Live from New York).mp3" of=/dev/null bs=1M
```

=====

AWS:

```
time dd if="/home/ubuntu/navidrome/music/OPE/Isn't She Lovely.mp3" of=/dev/null bs=1M
```

Fase 2: Medición inmediata de velocidad en caliente utilizando el archivo ya residente en RAM (Page Cache). Debe ser notablemente más rápido: el archivo ya está en la page cache (RAM) y no necesita ir al disco.

La lectura en frío tardará más tiempo y mostrará menos MB/s debido al acceso al disco físico. La lectura en caliente tardará prácticamente 0.00 segundos y la velocidad se disparará (GB/s) porque el archivo se lee directamente desde la RAM de Ubuntu.

Paso 2: Evaluación del Rendimiento Bajo Presión de Memoria (Con restricciones de Cgroups v2):

Repetición del ciclo experimental bajo los límites severos del cgroup establecidos previamente (128 MB). Registro de los cambios en los tiempos donde el kernel se ve forzado a matar páginas continuamente.

Para ejecutar esta fase, debes abrir dos terminales en paralelo a través de SSH:

I4. Terminal 1: Lanzamiento del Estímulo de Carga (Streaming en Vivo)

AWS EC2:

```
curl -s "http://localhost:4533/rest/search3?u=Ope2026&p=ope2026&v=1.16.1&c=miapp&query=<Parte del nombre de la canción>"
```

```
mplayer -ao null "http://localhost:4533/rest/stream?id=<id>&u=Ope2026&p=ope2026&v=1.16.1&c=miapp"
```

Máquina Virtual (salida de audio):

```
curl -s curl 'http://localhost:4533/rest/search3?query=Shawn&u=Ope2026&p=OPE2026&v=1.16.1&c=miapp&f=json'
```

```
mpv --cache=no --demuxer-max-bytes=100 "http://localhost:4533/rest/stream?id=7fH11doysdXoRqQvg9zVTs&u=Ope2026&p=OPE2026&v=1.16.1&c=miapp"
```

I5. Terminal 2: Captura de Telemetría e Instrumentación del Kernel:

Mientras la Terminal 1 está reproduciendo el flujo, ejecuta consecutivamente estos comandos en la segunda consola para auditar cómo el Kernel se ve forzado a purgar páginas continuamente para no desbordar los 128 MB:

I6. Ver throughput (rendimiento) y uso del disco en tiempo real

```
sudo apt install sysstat
ostat -dx 1
```

Monitoreo simultáneo en tiempo real de la saturación y rendimiento de los canales de disco. El disco sigue trabajando en la segunda vuelta porque el contenedor no puede retener caché.

I7. Ver qué proceso está generando el I/O

```
sudo apt install iotop
sudo apt install iotop-c
sudo iotop -o
```

Identificación de los Identificadores de Procesos exactos que consumen ancho de banda de E/S de bloques. Aquí se confirma visualmente que el proceso de Navidrome está generando I/O síncrono en disco.

I8. Medir la lectura en CALIENTE (repetir el mismo comando, sin vaciar caché)

```
free -h
```

Monitorear la mutación métrica en las columnas globales de memoria, enfocándose en "buff/cache". Permite observar el comportamiento y estancamiento de la caché asignada en el Host.